



ISEC3004 / CCSEP

Security Engineering — Group Assignment 1

VULNERABILITY RESEARCH REPORT

Web Application Security Analysis

NoSQL Injection & Path Traversal — Vulnerable Design, Evidence & Mitigation

AUTHOR (VULNERABLE APP DEV)

Aadhil Rizwan

ASSIGNMENT

CCSEP Assignment 1

DATE

September 2026

CLASSIFICATION

Academic — Internal Use

EXPLOITATION LEAD

Kav

MITIGATION LEAD

Shaheem

DETECTION LEAD

Sachin

INTEGRATION / EDITING

Bilal

NoSQL Injection — POST /api/auth/login

Path Traversal — GET /api/documents/view

CCSEP Assignment 1 — Security Vulnerability Research

Confidential — Academic Use Only

Table of Contents

1	Introduction & Project Overview	3
2	Environment & Setup	4
	2.1 Technology Stack	4
	2.2 Repository Layout	4
	2.3 Prerequisites & Installation	5
	2.4 Seed Accounts & Demo Files	5
3	Vulnerable Code Design — NoSQL Injection	6
	3.1 Feature Description & Intended Data Flow	6
	3.2 Vulnerable Code Snippet & Insecure Lines	7
	3.3 Why the Input Handling is Unsafe	8
4	Vulnerable Code Design — Path Traversal	9
	4.1 Feature Description & Intended Directory Boundary	9
	4.2 Vulnerable Code Snippet & Insecure Lines	10
	4.3 Why the Path Handling Fails to Enforce the Boundary	11
5	Code Comments & Design Notes	12
6	Normal Behaviour Evidence	14

SECTION 1

Introduction & Project Overview

Purpose, scope, and structure of the vulnerable application baseline.

This report documents the design and implementation of the intentionally vulnerable web application baseline developed for CCSEP Assignment 1. The application was built to host and demonstrate two real-world vulnerability classes — **NoSQL Query Selector Injection** and **Path Traversal (Directory Traversal)** — in a controlled, reproducible lab environment.

The codebase is structured as a Node.js/Express API backed by MongoDB, serving both a login authentication endpoint and a document archive service. Each vulnerability is self-contained and independently reproducible. The application auto-seeds test accounts on first boot, requires no external configuration, and falls back to an in-memory database if local MongoDB is unavailable — making it fully portable across group members' machines.

2

Vulnerabilities Hosted

2

Vulnerable Endpoints

3

Seed Test Accounts

6

Normal-Use Test Cases

Scope of This Report

The content in this report covers the following areas, all authored by Aadhil Rizwan (Vulnerable Application Development Lead) and submitted to Bilal for integration:

- Vulnerable code design for both vulnerability classes
- Annotated code snippets with exact insecure lines identified
- Function-level design notes and security risk commentary
- Full environment and setup documentation
- Verified normal-behaviour evidence (pre-attack baseline)



Academic Context

All credentials, keys, flags, and secrets in this application are fabricated for the purposes of demonstrating the vulnerability. No real production systems or data are involved.

Team Roles

Member	Role	Contribution to This Report
Aadhil Rizwan	Vulnerable Application Development Lead	Designed and built both vulnerable apps; authored all content in this report
Kav	Exploitation Lead	Executes attacks against the vulnerable baseline
Shaheem	Mitigation Lead	Produces the mitigated application version
Sachin	Detection Lead	Builds detection/tracing analysis
Bilal	Integration / Editor	Assembles all sections into the final report

SECTION 2

Environment & Setup

Technology stack, repository structure, prerequisites, and run instructions.

2.1 Technology Stack

Layer	Technology	Version / Notes
Runtime	Node.js	v18+ (v20 recommended)
Web Framework	Express	^4.21.2
Database	MongoDB via Mongoose	^8.9.5 — local or in-memory fallback
In-Memory DB	mongodb-memory-server	^10.1.3 — auto-started if no local Mongo
HTTP Logging	Morgan	^1.10.0
CORS	cors	^2.8.5
Env Config	dotenv	^16.4.7
Package Manager	npm	v9+
Shell	zsh / bash	macOS / Linux compatible

2.2 Repository Layout

CCSEP_Assignment-1/ tree

```
├── nosql-injection/
│   └── vulnerable/
│       ├── server.js           ← App entrypoint
│       ├── routes/auth.js      ← 🚩 Vulnerable login handler
│       ├── models/User.js      ← Mongoose User schema
│       ├── config/db.js        ← MongoDB connection manager
│       ├── config/.env.secrets ← Simulated secrets file (traversal target)
│       ├── scripts/seed.js     ← DB seed script
│       ├── scripts/verify_normal.sh ← Normal-use test runner
│       ├── public/             ← Static HTML/CSS/JS frontend
│       └── package.json
├── path-traversal/
│   └── vulnerable/
│       └── server.js           ← App entrypoint
```

```
| routes/documents.js ← Vulnerable document handler
| routes/auth.js      ← Login handler
| config/.env.secrets ← Traversal target file
| public/documents/   ← Permitted document directory
|   audit_report_2026.txt
|   security_policy_v2.txt
|   network_guidelines.txt
| package.json
| shared/
|   evidence/
|     normal_use_evidence.txt ← Verified normal-use output
| scripts/
|   start-vulnerable.sh ← Top-level startup helper
```

2.3 Prerequisites & Installation

No global installations beyond Node.js and npm are required. MongoDB is optional — the app falls back automatically to `mongodb-memory-server` if a local instance is not running.

NoSQL Injection App

```
Terminal - nosql-injection/vulnerable

$ cd nosql-injection/vulnerable
$ npm install
# Optional: explicitly seed test accounts
$ npm run seed
# Start the server
$ npm start
Server running at http://localhost:3000
[DB] Connected to in-memory database at mongodb://127.0.0.1: ...
[SERVER] Seed accounts initialized.
```

Path Traversal App

```
Terminal - path-traversal/vulnerable

$ cd path-traversal/vulnerable
$ npm install
$ npm start
Server running at http://localhost:3000
```



Port Conflict

Both apps run on port 3000 by default. Start only one at a time, or set `PORT=3001` as an environment variable before running the second.

2.4 Seed Accounts & Demo Files

Test Accounts (both applications)

Username	Password	Role	Department
alice_admin	AdminSecret#2026	Administrator	IT Security
bob_analyst	AuditPass!99	Security Analyst	SOC
charlie_guest	GuestWelcome123	Auditor	Compliance

Accounts are auto-seeded at startup if the User collection is empty. Re-seed at any time with `npm run seed`.

Permitted Documents (Path Traversal App)

Filename	Description	Classification
audit_report_2026.txt	Annual corporate ISMS audit summary	Internal Public
security_policy_v2.txt	Password and access control policy (v2.4)	Internal Public
network_guidelines.txt	Subnet topology and standard ports	Internal Public

Traversal Target File

The simulated secrets file used as the attack target for the path traversal demonstration is located at:

config/.env.secretsenv

```
# Simulated production secrets – for academic demonstration only
APP_SECRET_KEY="ISEC3004{fl4g_d1r3ct0ry_tr4v3rs4l_succ3ss_demo}"
JWT_SIGNING_KEY="super_secret_jwt_hmac_sha256_token_signing_key_2026"
PAYMENT_GATEWAY_API_KEY="sk_live_demo_984729184719284719284"
DB_ADMIN_USER="corp_master_dba"
DB_ADMIN_PASSWORD="SuperSecretMasterPassword2026!#"

```

This file is deliberately placed outside the permitted documents directory to simulate a sensitive configuration file that should never be reachable via the document API.

SECTION 3

Vulnerable Code Design — NoSQL Injection

Feature description, data flow, vulnerable code, insecure lines, and root cause analysis.

3.1 Feature Description & Intended Data Flow

The login feature allows corporate users to authenticate against a MongoDB user collection by submitting a JSON body containing their `username` and `password`. On a successful credential match, the server returns a user profile object and a session token. On failure, it returns HTTP 401 with a generic error message.

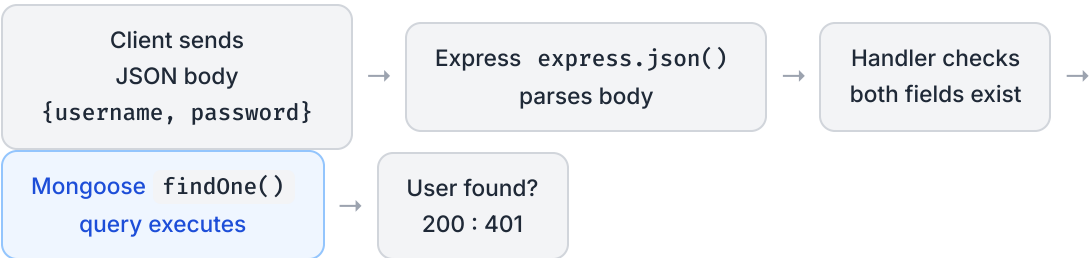
Endpoint

POST

/api/auth/login

routes/auth.js

Intended Data Flow (Normal Operation)



In the intended flow, both `username` and `password` should be plain strings that MongoDB compares using equality. The query is equivalent to: `SELECT * FROM users WHERE username = ? AND password = ?`

3.2 Vulnerable Code Snippet

nosql-injection/vulnerable/routes/auth.js — POST /api/auth/login

JavaScript

```
// POST /api/auth/login
router.post('/login', async (req, res) => {
  const { username, password } = req.body; // Line 24

  // Audit logging
  const clientIp = req.headers['x-forwarded-for'] || req.socket.remoteAddress;
  console.log(`[AUTH] Input types: username=[${typeof username}], password=[${typeof password}]`);

  // Undefined check — does NOT enforce string type
  if (username === undefined || password === undefined) {
    return res.status(400).json({ status: 'error', message: 'Both fields required.' });
  }
});
```

```
try {
  // -----
  // ⚠ INSECURE — Lines 44-47: queryFilter is built directly from req.body

  // values WITHOUT enforcing String type. If the client sends an object,
  // MongoDB receives it as a query operator, not a literal value.

  const queryFilter = {

    username: username,      // could be: "alice_admin" ← normal

    password: password      // could be: {"$ne": ""} ← injection

  };

  // -----

  console.log('[AUTH] Executing query:', JSON.stringify(queryFilter));

  const user = await User.findOne(queryFilter); // Line 51

  if (!user) {
    return res.status(401).json({ status: 'fail', message: 'Invalid username or password.' });
  }

  return res.status(200).json({
    status: 'success',
    data: { userId: user._id, username: user.username, role: user.role, ... }
  });
} catch (error) {
  return res.status(500).json({ status: 'error', message: 'Internal server error.' });
}
});
```

Insecure Lines — Summary Table

Line(s)	Code	Issue
44-47	queryFilter = { username, password }	Raw req.body values are placed directly into the Mongoose filter object. No type coercion or validation is applied. If either value is a JavaScript object, Mongoose passes it directly to MongoDB as a query operator.
33-36	if (username === undefined)	Only checks that the field is not undefined . An object such as {"\$ne": ""} passes this check. A correct guard would additionally assert typeof password === 'string' .

3.3 Why the Input Handling is Unsafe

Express is configured with the `express.json()` middleware (see `server.js`). This middleware fully deserialises the HTTP request body as a JavaScript value — including nested objects. When a client sends the body below, Express produces a JavaScript object where `password` is not a string, but a plain object containing the MongoDB query operator `$ne` (not-equal):

```
Attack payload — curl

$ curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "alice_admin", "password": {"$ne": ""}}'
```

What MongoDB Sees

Because `queryFilter.password` is the object `{"$ne": ""}`, Mongoose forwards the following query to MongoDB:

```
MongoDB query — operator injection MongoDB

// Intended query (string equality):
db.users.findOne({ username: "alice_admin", password: "AdminSecret#2026" })

// Injected query (operator object — password is NEVER checked for equality):
db.users.findOne({ username: "alice_admin", password: { $ne: "" } })

// → Matches any document where username = "alice_admin"
// → AND password IS NOT equal to "" (i.e., any non-empty password)
// → alice_admin's actual password is NEVER compared — query always succeeds
```

Root Cause Chain

1. `express.json()` deserialises nested JSON objects from the request body.
2. The handler only checks `password !== undefined` — an object is not undefined.
3. The object is placed directly into the Mongoose `findOne()` filter.
4. Mongoose passes the object to the MongoDB driver without coercing it to a string.
5. MongoDB interprets `{"$ne": ""}` as a query operator, bypassing the password check entirely.



Impact: Complete Authentication Bypass

An attacker who knows a valid username can log in as any user without knowing their password. Because the `$ne: ""` payload matches any document where the password field is non-empty, the attack succeeds against every account in the database.

What the Fix Looks Like

The handler should coerce both values to strings before constructing the query filter, rejecting any request where either value is not a primitive string:

Mitigation sketch (for Shaheem)

JavaScript

```
// Enforce string type – reject if not a primitive string
if (typeof username !== 'string' || typeof password !== 'string') {
  return res.status(400).json({ message: 'Invalid input type.' });
}

// Now safe – MongoDB receives a literal string for both fields
const user = await User.findOne({
  username: String(username).trim(),
  password: String(password)
});
```

SECTION 4

Vulnerable Code Design — Path Traversal

Feature description, intended directory boundary, vulnerable code, and root cause analysis.

4.1 Feature Description & Intended Directory Boundary

The document archive service allows authenticated users to read compliance and audit reports stored in a designated `public/documents/` directory. The client passes a filename via a query string parameter and the server responds with the file's contents as plain text or JSON.

Endpoint

GET

/api/documents/view?file=<filename>

routes/documents.js

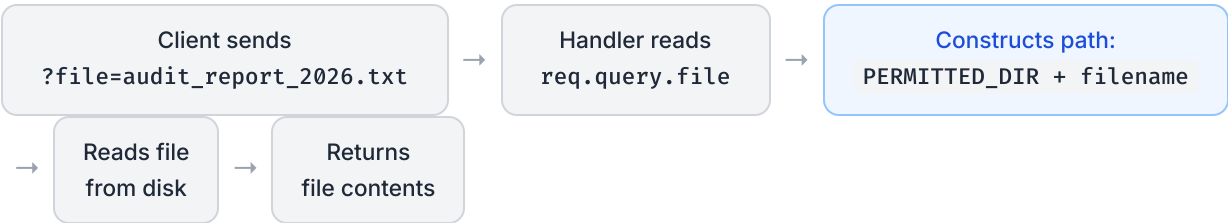
Intended Directory Boundary

routes/documents.js — base directory constant

JavaScript

```
// Intended base directory — ONLY files inside this path should ever be served
const PERMITTED_DOCUMENT_DIR = path.join(__dirname, '../public/documents');
// Resolves to: /path/to/path-traversal/vulnerable/public/documents/
```

Intended Data Flow (Normal Operation)



In the intended flow, a filename like `audit_report_2026.txt` resolves to `.../public/documents/audit_report_2026.txt` — entirely within the permitted base directory.

4.2 Vulnerable Code Snippet

path-traversal/vulnerable/routes/documents.js — GET /api/documents/view

JavaScript

```
const PERMITTED_DOCUMENT_DIR = path.join(__dirname, '../public/documents');

router.get('/view', (req, res) => {
  const fileName = req.query.file; // Line 43 — raw, unvalidated user input

  if (!fileName) {
```

```
    return res.status(400).json({ message: "Missing 'file' query parameter." });
  }

  // -----
  // ⚠ INSECURE – Line 54: path.join() normalises '../' sequences but does
  // NOT verify that the final resolved path stays within PERMITTED_DOCUMENT_DIR.
  // Supplying '../../config/.env.secrets' escapes the base directory.

  const targetFilePath = path.join(PERMITTED_DOCUMENT_DIR, fileName);

  // -----

  // Request tracing – note: traversal flag is LOGGED but NOT BLOCKED
  const isTraversalAttempt = fileName.includes('..') || fileName.includes('/');
  console.log(`[DOC] Traversal flag: ${isTraversalAttempt}`); // Log-only, never blocks

  // File is read from the resolved (potentially escaped) path – no boundary check
  fs.readFile(targetFilePath, 'utf8', (err, data) => {
    if (err) {
      return res.status(404).json({ message: 'File not found.' });
    }
    return res.status(200).send(data);
  });
});
```

Insecure Lines — Summary Table

Line	Code	Issue
54	path.join(PERMITTED_DOCUMENT_DIR, fileName)	path.join() normalises the path (resolves ../ tokens) but produces no error and performs no containment check. The resolved path can point anywhere on the filesystem that the Node process has read access to.
57-58	isTraversalAttempt flag	The traversal attempt is detected and logged, but this is the intended design of the vulnerable application — the detection is deliberate evidence for the tracing section. In a real application, detection without blocking would be a second vulnerability.

4.3 Why the Path Handling Fails to Enforce the Boundary

Node's `path.join()` function is a path-construction utility, not a security boundary. It resolves and normalises relative path tokens — including `../` (parent directory traversal) — producing a clean absolute path. It does not know or care whether the resulting path is inside the intended base directory. There is no exception thrown, no error returned, and no indication in the output that the boundary has been crossed.

Path Resolution Example

Node.js — `path.join()` behaviour with traversal tokens

Node REPL

```
// PERMITTED_DOCUMENT_DIR resolves to:
// /path/to/vulnerable/public/documents

// Normal request — stays inside boundary:
path.join('/path/to/vulnerable/public/documents', 'audit_report_2026.txt')
// → '/path/to/vulnerable/public/documents/audit_report_2026.txt' ✓ SAFE

// Traversal attack — escapes boundary completely:
path.join('/path/to/vulnerable/public/documents', '../../config/.env.secrets')
// → '/path/to/vulnerable/config/.env.secrets' ✗ OUTSIDE PERMITTED DIR

// path.join() silently normalises the '../' — no error, no warning
```

Attack Payload

Attack payload — curl

```
$ curl "http://localhost:3000/api/documents/view?file=../../config/.env.secrets"

# Server response — sensitive file contents leaked:
APP_SECRET_KEY="ISEC3004{fl4g_d1r3ct0ry_tr4v3rs4l_succ3ss_demo}"
JWT_SIGNING_KEY="super_secret_jwt_hmac_sha256_token_signing_key_2026"
PAYMENT_GATEWAY_API_KEY="sk_live_demo_984729184719284719284"
DB_ADMIN_USER="corp_master_dba"
DB_ADMIN_PASSWORD="SuperSecretMasterPassword2026!#"
```

Root Cause Chain

1. `req.query.file` accepts arbitrary user input, including `../` sequences.
2. `path.join()` normalises the input, producing a valid absolute path outside the permitted directory.
3. The `isTraversalAttempt` check logs the traversal but does not reject the request.

4. `fs.readFile()` reads whatever file is at the resolved path — no boundary enforcement exists at the filesystem read stage.

**Impact: Arbitrary File Read**

An attacker can read any file accessible to the Node.js process — configuration files, private keys, environment variables, or OS files — by constructing a path with enough `../` tokens to climb to the target file.

What the Fix Looks Like

Mitigation sketch (for Shaheem)

JavaScript

```
const targetFilePath = path.resolve(PERMITTED_DOCUMENT_DIR, fileName);

// Enforce boundary: resolved path MUST start with permitted dir
if (!targetFilePath.startsWith(PERMITTED_DOCUMENT_DIR + path.sep)) {
  return res.status(403).json({ message: 'Access denied: path is outside permitted directory.'
}
```

SECTION 5

Code Comments & Design Notes

Function-by-function walkthrough of what each component does and the risk each vulnerable implementation introduces.

5.1 `routes/auth.js` — Authentication Handler

Purpose: Handles `POST /api/auth/login`. Reads `username` and `password` from the JSON request body, runs a Mongoose `findOne()` query against the `User` collection, and returns user profile data and a session token on a successful match.

Design decision — why no string enforcement: The handler intentionally omits the `typeof username === 'string'` guard. The only validation present is an `undefined` check on lines 33–36, which allows any truthy non-undefined value — including objects — to pass through to the query filter.

Risk introduced: Any client that can send JSON with a nested object for the `password` field (or `username` field) can inject a MongoDB query operator, bypassing authentication entirely.

5.2 `routes/documents.js` — Document Archive Handler

Purpose: Handles `GET /api/documents/view?file=`. Reads a filename from the query string, constructs the target file path using `path.join()`, and streams the file back to the client as plain text or JSON.

Design decision — why `path.join()` without boundary check: The handler uses the standard path utility but omits a `path.resolve() + startsWith()` boundary check. The `isTraversalAttempt` flag on line 57 detects `..` in the filename and logs a warning, but deliberately does not reject the request — this is intentional for the tracing/detection section of the assignment.

Risk introduced: An attacker can navigate to any directory the process can read by supplying `../` tokens in the `file` parameter, leaking configuration files, secrets, and other sensitive resources.

5.3 `routes/documents.js` — Document Listing Handler

Purpose: Handles `GET /api/documents/list`. Reads the `public/documents/` directory with `fs.readdirSync()` and returns the list of `.txt` files as JSON. This endpoint is safe — it reads a fixed, hardcoded directory and exposes only filenames, not file contents.

Risk introduced: None. This endpoint is informational and operates only on the pre-validated directory constant.

5.4 `config/db.js` — Database Connection Manager

Purpose: Manages MongoDB connectivity. On startup, it attempts to connect to a local MongoDB instance at `mongodb://127.0.0.1:27017/ise3004_vulnerable_db`. If the connection fails within 2 seconds (e.g., MongoDB is not installed locally), it automatically falls back to an in-memory `mongodb-memory-server` instance and logs the fallback URI.

Design decision: The fallback makes the application fully portable — any group member can run it without installing MongoDB. The in-memory instance is seeded with the same test accounts as a local database via the `autoSeedIfEmpty()` function.

Risk introduced: None. The database connection module has no user-facing attack surface.

5.5 `server.js` — Application Entrypoint

Purpose: Bootstraps the Express application. Registers the following middleware stack in order: `cors()`, `express.json()`, `express.urlencoded({ extended: true })`, `morgan` (HTTP logging), and `express.static()` for the frontend. Mounts route handlers under `/api/auth` and (in the path traversal app) `/api/documents`. Calls `autoSeedIfEmpty()` after database connection to ensure test accounts are always present.

Design note — `express.json()`: Registering `express.json()` globally is a necessary enabler of the NoSQL injection vulnerability. This middleware fully deserialises JSON objects from the request body, including nested objects, which is what allows operator injection payloads to reach the query filter as live JavaScript objects rather than strings.

5.6 `models/User.js` — Mongoose User Schema

Purpose: Defines the Mongoose schema for user documents. Fields include `username` (unique, trimmed string), `password` (plain string — no hashing, by design), `fullName`, `role` (enum: Administrator, Security Analyst, Auditor, Staff), `department`, `accountStatus` (Active/Suspended), and `createdAt`.

Design decision — plain-text passwords: Passwords are stored as plain strings. This is intentional — the assignment focuses on the query-level injection vulnerability, not the storage vulnerability. Introducing bcrypt would add complexity without contributing to the demonstrated vulnerability class.

Risk introduced: The schema's `password` field is typed as `String` in Mongoose, but Mongoose's type coercion is bypassed when an object is passed directly as a query filter value (not as a document being saved). The schema type enforcement only applies during document creation and updates, not during `findOne()` query filters.

5.7 `scripts/seed.js` — Database Seed Script

Purpose: Clears the User collection and inserts three test accounts. Invoked manually with `npm run seed` or automatically at startup if the collection is empty. Connects to the database, deletes all existing documents, inserts the seed array, and disconnects cleanly.

Design note: The seed script provides a known, reproducible database state for demonstration and exploitation. The same three accounts are used across all phases of the assignment (normal use, exploitation, mitigation testing).

5.8 `scripts/verify_normal.sh` — Normal-Use Test Runner

Purpose: A bash script that exercises six normal-use scenarios using `curl` and writes the results to `docs/evidence/normal_use_evidence.txt`. The tests cover: health endpoint, valid authentication (alice_admin), invalid authentication (wrong password), two valid document retrievals, and a 404 for a non-existent document.

Design note: This script is the primary evidence source for Section 6 of this report. It establishes the baseline that confirms both application features work correctly before exploitation begins.

SECTION 6

Normal Behaviour Evidence

Verified pre-attack output showing both features function correctly before exploitation. All results captured by `scripts/verify_normal.sh` on 2026-09-19 17:49 UTC.

✓

All 6 Normal-Use Tests Passed

Evidence captured against `http://127.0.0.1:3000` running the combined vulnerable baseline application. Source: `shared/evidence/normal_use_evidence.txt`

Test 1 — Health Endpoint Check

Confirms the server is running and both vulnerability endpoints are registered and accessible before any attack is attempted.

GET /api/health

200 OK

Request	GET http://127.0.0.1:3000/api/health
Expected	HTTP 200 — system online
Result	PASS

Captured output — verify_normal.sh

```
--- TEST 1: Health Endpoint Check ---
HTTP Status: 200
Response Body:
{
  "status": "online",
  "system": "ISEC3004 Vulnerable Baseline Portal",
  "version": "1.0.0-vulnerable-baseline",
  "vulnerabilitiesHosted": [
    { "name": "NoSQL Injection", "endpoint": "POST /api/auth/login",
"vulnerableFile": "routes/auth.js:58" },
    { "name": "Path Traversal", "endpoint": "GET /api/documents/view",
"vulnerableFile": "routes/documents.js:74" }
  ],
  "timestamp": "2026-09-19T17:49:44.376Z"
}
```

Test 2 — Valid Authentication (alice_admin)

Demonstrates that the login endpoint works correctly when legitimate credentials are supplied — a valid username and the matching plaintext password resolve to a successful 200 response with the user's profile and session token.

POST /api/auth/login — valid credentials

200 OK

Request

POST /api/auth/login
{"username":"alice_admin","password":"AdminSecret#2026"}

Expected

HTTP 200 — returns user profile and token

Result

PASS

Captured output — verify_normal.sh

```
--- TEST 2: Valid Authentication (alice_admin) ---
Request: POST /api/auth/login
HTTP Status: 200
Response Body:
{
  "status": "success",
  "message": "Authentication successful.",
  "data": {
    "userId": "6aa5a92e9b9e9e851d8260d1",
    "username": "alice_admin",
    "fullName": "Alice Sterling (System Administrator)",
    "role": "Administrator",
    "department": "Executive IT Security",
    "token": "session_token_6aa5a92e9b9e9e851d8260d1_1789840184452"
  }
}
```

Test 3 — Invalid Authentication (Wrong Password)

Confirms that the login endpoint correctly rejects an incorrect password with HTTP 401. This establishes the security contract: a wrong string password fails. The attack in Section 3 bypasses this check by supplying an operator object instead of a string.

POST /api/auth/login — wrong password

401 Unauthorised

Request

POST /api/auth/login
{"username":"alice_admin","password":"WrongPassword999!"}

Expected

HTTP 401 — rejected

Result

PASS

Captured output — verify_normal.sh

```
--- TEST 3: Invalid Authentication (Rejection) ---  
Request: POST /api/auth/login  
HTTP Status: 401  
Response Body:  
{  
  "status": "fail",  
  "message": "Invalid username or password."  
}
```

Test 4 — Valid Document Retrieval (audit_report_2026.txt)

Confirms that a legitimate permitted filename retrieves the correct document with HTTP 200. This baseline shows the document service works before the traversal payload is applied.

GET /api/documents/view?file=audit_report_2026.txt **200 OK**

Request	GET /api/documents/view?file=audit_report_2026.txt
Expected	HTTP 200 – returns audit report content
Result	PASS

```
Captured output – verify_normal.sh

--- TEST 4: Valid Document Retrieval (audit_report_2026.txt) ---
Request: GET /api/documents/view?file=audit_report_2026.txt
HTTP Status: 200
Document Preview:

ISEC3004 ENTERPRISE SECURITY AUDIT REPORT - 2026
Permitted Demonstration Document (Classification: Internal Public)

Audit Scope: Corporate Information Security Management System (ISMS)
Conducted By: Independent Security Audit Team – Date: September 2026
```

Test 5 — Valid Document Retrieval (security_policy_v2.txt)

A second legitimate document retrieval test confirming the document endpoint serves multiple permitted files correctly.

GET /api/documents/view?file=security_policy_v2.txt **200 OK**

Request	GET /api/documents/view?file=security_policy_v2.txt
Expected	HTTP 200 – returns security policy content
Result	PASS

```
Captured output – verify_normal.sh

--- TEST 5: Valid Document Retrieval (security_policy_v2.txt) ---
Request: GET /api/documents/view?file=security_policy_v2.txt
HTTP Status: 200
Document Preview:
```

CORPORATE INFORMATION SECURITY POLICY (VERSION 2.4)

Permitted Demonstration Document (Classification: Internal Public)

Effective Date: January 1, 2026 — Document Owner: CISO

Test 6 — Non-Existent Document (404 Rejection)

Confirms the document endpoint returns HTTP 404 when a filename that does not exist is requested. This demonstrates that the server applies proper error handling for genuinely missing files — distinguishing a normal 404 from a traversal attack that reaches an existing file outside the permitted directory.

GET /api/documents/view?file=missing_record_xyz.txt

404 Not Found

Request

GET /api/documents/view?file=missing_record_xyz.txt

Expected

HTTP 404 — file not found

Result

PASS

Captured output — verify_normal.sh

--- TEST 6: Non-Existent Document (404 Rejection) ---
Request: GET /api/documents/view?file=missing_record_xyz.txt
HTTP Status: 404
Response Body:
{
 "status": "fail",
 "message": "Requested document not found."
}

Normal-Use Test Summary

#	Test	Endpoint	Expected Status	Actual Status	Result
1	Health check	GET /api/health	200	200	PASS
2	Valid login	POST /api/auth/login	200	200	PASS
3	Wrong password	POST /api/auth/login	401	401	PASS
4	Valid doc retrieval	GET /api/documents/view	200	200	PASS

#	Test	Endpoint	Expected Status	Actual Status	Result
5	Valid doc retrieval #2	GET /api/documents/view	200	200	PASS
6	Missing file 404	GET /api/documents/view	404	404	PASS

✓

6 / 6 tests passed — Baseline Verified

Both vulnerable features operate correctly under normal conditions. The application is confirmed ready for exploitation testing (Kav) and detection analysis (Sachin). Evidence file: `shared/evidence/normal_use_evidence.txt` — captured 2026-09-19 17:49 UTC.